

```
import tkinter as tk

from tkinter import messagebox

import pandas as pd

from sklearn.ensemble import RandomForestClassifier

import joblib

import os

import numpy as np

import matplotlib.pyplot as plt

import arabic_resaper

from bidi.algorithm import get_display


class DiabetesPredictorApp:

    def __init__(self, root):

        self.root = root

        self.root.title("Neelain University - Master's Project")

        self.root.geometry("750x900")

        self.root.configure(bg="#f4f7f6")


        self.data_path = r"diabetes_data.csv"

        self.model_filename = "diabetes_model.pkl"


        self.features_info =

        """
        en": "Pregnancies", "ar": "عدد مرات الحمل", "range": "0 - 17", "ideal": "0,{
        en": "Glucose", "ar": "مستوى الجلوكوز", "range": "70 - 200", "ideal": "70-100,{
        en": "BloodPressure", "ar": "ضغط الدم", "range": "40 - 122", "ideal": "80,{
        en": "SkinThickness", "ar": "سمك الجلد", "range": "0 - 99", "ideal": "20,{
        """
```

```

"}      en": "Insulin", "ar", "الإنسولين": "range": "0 - 846", "ideal": "79,{
"}      en": "BMI", "ar", "مؤشر كتلة الجسم": "range": "10 - 67", "ideal": "18.5-25,{
"}      en": "DiabetesPedigreeFunction", "ar", "وراثة السكري": "range": "0.08 - 2.4", "ideal": "0.5,{
"}      en": "Age", "ar", "العمر": "range": "21 - 81", "ideal{---": "
[

```

```

self.inputs{} =

```

```

self.model = None

```

```

self.result_var = tk.StringVar()

```

```

self.result_var.set(self.fix_text(("أدخل البيانات ثم اضغط على زر التحليل")))

```

```

self.prepare_system()

```

```

self.create_widgets()

```

```

def fix_text(self, text):

```

```

    reshaped_text = arabic_reshaper.reshape(text)

```

```

    return get_display(reshaped_text)

```

```

def prepare_system(self):

```

```

    # محاولة تحميل النموذج أو تدريبه

```

```

    if os.path.exists(self.model_filename):

```

```

        try:

```

```

            self.model = joblib.load(self.model_filename)

```

```

        except:

```

```

            self.train_model_logic()

```

```

    else:

```

```

        self.train_model_logic()

```

```
def train_model_logic(self):
```

```
    if os.path.exists(self.data_path):
```

```
        try:
```

```
            df = pd.read_csv(self.data_path)
```

```
            features_en = [f['en'] for f in self.features_info]
```

```
            X = df[features_en]
```

```
            y = df['Outcome']
```

```
            self.model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
            self.model.fit(X.values, y)
```

```
            joblib.dump(self.model, self.model_filename)
```

```
        except Exception as e:
```

```
            print("ERROR:", e)
```

```
            messagebox.showerror("ERROR", str(e))
```

```
def create_widgets(self):
```

```
    # الهوية والأسماء
```

```
    header_frame = tk.Frame(self.root, bg="#1a4d2e", pady=5)
```

```
    header_frame.pack(fill=tk.X)
```

```
    tk.Label(header_frame, text=self.fix_text, ("جامعة النيلين")font=("Tahoma", 16, "bold"),  
bg="#1a4d2e", fg="#deff9a").pack()
```

```
    info_frame = tk.Frame(self.root, bg="#2e7d32", pady=5)
```

```
    info_frame.pack(fill=tk.X)
```

```
    tk.Label(info_frame, text=self.fix_text, ("تصميم وبرمجة طالب ماجستير : محمد ابوالقاسم")font=("Tahoma",  
10, "bold"), bg="#2e7d32", fg="white").pack()
```

```
    tk.Label(info_frame, text=self.fix_text, ("إشراف دكتور : امين مبارك")font=("Tahoma", 12, "bold"),  
bg="#2e7d32", fg="#deff9a").pack()
```

```
    # تم تكبيرها لضمان الظهور في منطقة النتيجة
```

```
result_frame = tk.LabelFrame(self.root, text=self.fix_text, ("نتائج التحليل")font=("Tahoma", 11, "bold"), bg="#ffffff", fg="white", labelanchor="ne("
```

```
result_frame.pack(fill=tk.X, padx=40, pady=10)
```

```
self.result_label = tk.Label(result_frame, textvariable=self.result_var, font=("Tahoma", 11, "bold"), bg="#ffffff", fg="#333", height=4, wraplength=600, justify="right")
```

```
self.result_label.pack(pady=10)
```

```
container = tk.Frame(self.root, bg="#f4f7f6", padx=30)
```

```
container.pack(expand=True, fill=tk.BOTH)
```

```
# جدول المدخلات
```

```
table_header = tk.Frame(container, bg="#cfd8dc")
```

```
table_header.pack(fill=tk.X, pady=5)
```

```
tk.Label(table_header, text=self.fix_text, ("المعيار الطبي")font=("Tahoma", 9, "bold"), bg="#cfd8dc", width=25.(pack(side=tk.RIGHT)
```

```
tk.Label(table_header, text=self.fix_text, ("القيمة")font=("Tahoma", 9, "bold"), bg="#cfd8dc", width=15.(pack(side=tk.RIGHT)
```

```
tk.Label(table_header, text=self.fix_text, ("المدى المثالي")font=("Tahoma", 9, "bold"), bg="#cfd8dc", width=20.(pack(side=tk.RIGHT)
```

```
for info in self.features_info:
```

```
frame = tk.Frame(container, bg="#f4f7f6")
```

```
frame.pack(fill=tk.X, pady=2)
```

```
tk.Label(frame, text=self.fix_text(info['ar']), font=("Tahoma", 10), bg="#f4f7f6", width=25, anchor="e").pack(side=tk.RIGHT)
```

```
entry = tk.Entry(frame, font=("Arial", 11), justify="center", bd=2, width=15)
```

```
entry.pack(side=tk.RIGHT, padx=10)
```

```
self.inputs[info['en']] = entry
```

```
tk.Label(frame, text=f"{info['range']} ({info['ideal']})", font=("Arial", 8), fg="#546e7a",
bg="#f4f7f6", width=20).pack(side=tk.RIGHT)
```

#الأزرار

```
btn_container = tk.Frame(self.root, bg="#f4f7f6", pady=15)
```

```
btn_container.pack()
```

```
tk.Button(btn_container, text=self.fix_text, ("تحليل الحالة")font=("Tahoma", 12, "bold"),
bg="#2e7d32", fg="white", padx=50, pady=10, command=self.predict_diabetes.grid(row=0,
column=1, padx=10)
```

```
tk.Button(btn_container, text=self.fix_text, ("تحليل العوامل")font=("Tahoma", 10), bg="#455a64",
fg="white", padx=20, command=self.show_importance_chart.grid(row=0, column=0, padx=10)
```

```
def predict_diabetes(self):
```

```
if not self.model:
```

```
self.result_var.set(self.fix_text "C:\proj\\diabetes_data.csv(("خطأ: تأكد من ملف
```

```
self.root.update_idletasks فوراً # تحديث الواجهة فوراً
```

```
return
```

```
try:
```

```
features_en = [f['en'] for f in self.features_info]
```

```
data = [float(self.inputs[f].get()) for f in features_en]
```

```
prediction = self.model.predict([data])[0]
```

```
prob = self.model.predict_proba([data])[0][1] * 100
```

```
if prediction == 1:
```

```
res = f\ "التشخيص: احتمال وجود إصابة بالسكري.\n"
توصية: يرجى مراجعة {prob:.1f}%\n
المختبر لإجراء الفحوصات."
```

```
self.result_label.config(fg="#c62828")
```

```
else:
```

```
res = f\ "التشخيص: الحالة سليمة من السكري حالياً.\n"
توصية: حافظ على {100-prob:.1f}%\n
النشاط البدني."
```

```
self.result_label.config(fg="#2e7d32")
```

```
self.result_var.set(self.fix_text(res))
```

```
self.root.update_idletasks() # إجبار الملف التنفيذي على إظهار النتيجة
```

```
except ValueError:
```

```
self.result_var.set(self.fix_text(("خطأ: يرجى إدخال جميع الأرقام بشكل صحيح."))
```

```
self.root.update_idletasks()
```

```
def show_importance_chart(self):
```

```
    if not self.model: return
```

```
    importances = self.model.feature_importances_
```

```
    indices = np.argsort(importances)
```

```
    plt_labels = [self.fix_text(self.features_info[i]['ar']) for i in indices]
```

```
    plt.figure(figsize=(10, 6))
```

```
    plt.barh(range(len(indices)), importances[indices], color='#1a4d2e')
```

```
    plt.yticks(range(len(indices)), plt_labels)
```

```
    plt.title(self.fix_text(("تحليل المعايير الطبية"))
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
if __name__ == "__main__":
```

```
    root = tk.Tk()
```

```
    app = DiabetesPredictorApp(root)
```

```
    root.mainloop()
```

شرح مفصل للمشروع مقسم إلى أجزاء تقنية:

1. المكتبات المستخدمة (Libraries) :

- **tkinter**: المكتبة القياسية لإنشاء واجهة المستخدم الرسومية (GUI).
- **pandas**: تستخدم لمعالجة وتحليل البيانات (قراءة ملف CSV).
- **RandomForestClassifier**: خوارزمية تعلم الآلة المستخدمة. تعتمد على مبدأ "الغابات العشوائية" (Ensemble Learning) وهي قوية جداً في التصنيف الطبي.
- **joblib**: تستخدم لعملية (Serialization)، أي حفظ النموذج المدرب في ملف واستعادته لاحقاً دون الحاجة لإعادة التدريب.
- **bidirectional & arabic_reshaper**: ضروريان لمعالجة النصوص العربية، حيث تقومان بربط الحروف المقطعة وعكس اتجاه النص ليظهر بشكل صحيح.
- **matplotlib**: تستخدم لإنشاء الرسوم البيانية (Chart) التي توضح أهمية الخصائص الطبية.

2. هيكلية الكود (Class-Based Structure)

الكود مصمم باستخدام البرمجة كائنية التوجه (OOP) عبر كلاس DiabetesPredictorApp لضمان تنظيم الكود وسهولة صيانه.

أ- الدالة البانية `__init__`

- تقوم بتهيئة النافذة الرئيسية وضبط الألوان والمقاسات.
- تحتوي على قائمة `features_info` وهي قاموس بيانات (Dictionary) يربط بين المسميات البرمجية بالإنجليزية، والمسميات العربية، والقيم المثالية، مما يسهل عملية عرضها وتدريب النموذج.

ب- معالجة اللغة العربية `fix_text` :

هذه الدالة هي "المحرك" الذي يضمن عدم ظهور الحروف العربية كمربعات أو حروف مقلوبة، حيث تقوم بإعادة تشكيل الحروف وتنسيقها برمجياً قبل عرضها على الواجهة.

3. منطق ذكاء الآلة (Machine Learning Logic) :

ج- دالة `train_model_logic & prepare_system`

- **التحقق (Check)**: يبحث البرنامج عن ملف `diabetes_model.pkl`. إذا وجدته، يقوم بتحميله فوراً لتوفير الوقت.
- **التدريب (Training)**: إذا لم يجد النموذج، يتجه إلى ملف CSV ويستخدم خوارزمية **Random Forest** مع 100 شجرة قرار (`n_estimators=100`).
- **المتغيرات**: يتم فصل البيانات إلى X (المعايير الطبية) و y (النتيجة: مصاب أو سليم).

د- دالة التنبؤ `predict_diabetes` :

1. تأخذ المدخلات من المستخدم وتحولها إلى أرقام (float).

2. تستخدم model.predict لتحديد الحالة.

3. تستخدم model.predict_proba لحساب درجة الموثوقية (الاحتمالية الإحصائية للنتيجة)، وهو ما يعطي البرنامج طابعاً علمياً دقيقاً.

4. تحليل الأهمية (Feature Importance) :

دالة show_importance_chart هي ميزة متقدمة في مشروعك:

- تقوم باستخراج قيم feature_importances_ من خوارزمية Random Forest.
- توضح للدكتور أي العوامل (مثل الجلوكوز أو العمر) كانت الأكثر تأثيراً في اتخاذ القرار البرمجي.

5. واجهة المستخدم (User Interface) :

- تنسيق الجدول: تم استخدام حلقة تكرار (for loop) لإنشاء حقول الإدخال، مما يجعل الكود نظيفاً (Clean Code).
- التفاعلية: الألوان تتغير ديناميكياً (الأحمر للإصابة والأخضر للحالة السليمة) لتسهيل القراءة.
- التوافقية: إضافة self.root.update_idletasks() تضمن أن الواجهة ستستجيب وتعرض النتائج فوراً حتى لو كان البرنامج يعمل كمكلف تنفيذي مضغوط.

لماذا تم اختيار Random Forest؟

- الإجابة: "لأنها خوارزمية تجميعية (Ensemble) تقلل من مشكلة (Overfitting)، وتعطي نتائج دقيقة جداً مع البيانات الطبية الصغيرة والمتوسطة، كما أنها تسمح لنا باستخراج أهمية العوامل (Feature Importance) التي تؤثر على المرض."

أولاً: طريقة عمل خوارزمية Random Forest :

تعتمد هذه الخوارزمية على مبدأ يسمى Ensemble Learning (التعلم الجماعي)، وتحديدًا تقنية الـ Bagging (Bootstrap Aggregating).

- مبدأ "حكمة الجمهور": بدلاً من الاعتماد على شجرة قرار واحدة قد تخطئ، نقوم بإنشاء غابة مكونة من 100 شجرة (كما حددت في الكود n_estimators=100).
- العشوائية المزدوجة:

1. عشوائية البيانات: كل شجرة تتدرب على عينة عشوائية مختلفة من البيانات (Bootstrap).
2. عشوائية الخصائص: عند كل انقسام في الشجرة، تختار الخوارزمية مجموعة عشوائية من الميزات (مثلاً الجلوكوز والضغط فقط) لتفاضل بينهما، مما يمنع سيطرة ميزة واحدة على كل الأشجار.
- التصويت (Voting): عندما تُدخل بيانات مريض جديد، تقوم كل شجرة بإعطاء رأيها (مصاب أو سليم)، والنتيجة النهائية تكون بناءً على "الأغلبية".

ثانياً: شرح دالة prepare_system (إدارة دورة حياة النموذج) :

هذه الدالة هي المسؤول الإداري عن البرنامج، ووظيفتها تقليل استهلاك الموارد (Efficiency).

Python


```
def prepare_system(self):
    if os.path.exists(self.model_filename) :      # التحقق من وجود ملف النموذج
        try:
            self.model = joblib.load(self.model_filename)      # تحميل النموذج
        except:
            self.train_model_logic()
    else:
        self.train_model_logic()      # إذا فشل تحميل النموذج قم بتدريب نموذج جديد
```

- **Persistence (الاستمرارية):** الهدف منها هو عدم إعادة تدريب النموذج في كل مرة يفتح فيها المستخدم البرنامج. التدريب عملية تستهلك المعالج والوقت، لذا نقوم بـ "تجميد" الذكاء الذي اكتسبه البرنامج في ملف .pkl.
- **المرونة:** تسمح للبرنامج بالعمل فوراً في المناقشة حتى لو لم يكن هناك ملف بيانات CSV، طالما أن ملف الـ pkl موجود.

ثالثاً: شرح دالة **train_model_logic** (هندسة بناء النموذج):

هنا يحدث "التعلم الحقيقي". هذه الدالة هي المختبر الذي يحول الأرقام الجامدة إلى قواعد منطقية.

Python

```
def train_model_logic(self):
    if os.path.exists(self.data_path):      # التحقق من وجود ملف البيانات
        try:
            df = pd.read_csv(self.data_path)      # df اسمه DataFrame تخزين البيانات داخل
            features_en = [f['en'] for f in self.features_info]      # (Features) استخراج أسماء الخصائص
            X = df[features_en]
            y = df['Outcome']      # تحديد العمود المطلوب التنبؤ به
            # إنشاء نموذج Random Forest
            self.model = RandomForestClassifier(n_estimators=100, random_state=42)
            self.model.fit(X.values, y)      # تدريب النموذج
```

```
joblib.dump(self.model, self.model_filename) # حفظ النموذج بعد التدريب
```

```
except: # معالجة الأخطاء
```

```
pass
```

تفاصيل تقنية هامة:

```
self.model = RandomForestClassifier(n_estimators=100, random_state=42)
```

- **n_estimators=100**: تعني أننا بنينا 100 شجرة. زيادة هذا الرقم تزيد الدقة ولكن تجعل البرنامج أبطأ. 100 هو الرقم الذهبي (Sweet Spot) للتوازن.
- **random_state=42**: هذا الرقم يضمن أن النتائج ستكون "قابلة لإعادة الإنتاج" (Reproducible). أي أنك لو شغلت البرنامج في جهازك أو جهاز الدكتور، سيعطي نفس النتائج تماماً.

رابعاً: تعتمد قوة النموذج على :

1. **Robustness (المتانة)**: بفضل Random Forest، النظام لا يتأثر كثيراً بالبيانات الشاذة (Outliers).
2. **Explainability (القابلية للتفسير)**: البرنامج لا يعطي نتيجة صماء، بل يوفر "درجة موثوقية" (Probability) ويسمح برسم أهمية العوامل، وهو مطلب أساسي في الأنظمة الطبية.
3. **Optimization (الأمثلة)**: استخدام joblib لتحميل النموذج الجاهز بدلاً من التدريب المستمر .

إشراف : د. امين مبارك

اعداد طالب ماجستير : محمد ابوالقاسم